Final Project Report :AI Lecturer -Automated Course Content and Lecturer Video Generation

Venkata Susmitha, Duggineni
Data Science
University Of New Haven
vdugg2@unh.newhaven.edu

Srikanth, Thota
Data Science
University Of New Haven
sthot19@unh.newhaven.edu

Suryateja Reddy, Chitti
Data Science
University Of New Haven
schit25@unh.newhaven.edu

Anji Reddy, Sattaru
Data Science
University Of New Haven
asatt1@unh.newhaven.edu

Mounika , Aithagoni
Data Science
University Of New Haven
maith2@unh.newhaven.edu

Vedanth Reddy, Doddannagari
Data Science
University Of New Haven
vdodd7@unh.newhaven.edu

Hima Sai,Kuruba
Data Science
University Of New Haven
hkuru1@unh.newhaven.edu

Abstract: The AI Lecturer system is an innovative platform designed to automate the generation of educational content. It will create structured course materials, AI-generated lecture videos featuring animated avatars, and interactive exercises with adaptive assessments. This solution seeks to reduce the workload on educators while delivering a personalized learning experience.

Inspired by tools like Notebook LM, AI Lecturer leverages state-of-the-art AI to synthesize lecture notes, generate engaging video lectures, and adapt assessments based on learner profiles—resulting in a dynamic and fully automated educational experience.

Keywords: AI Lecturer, Educational Technology, Content Generation, AI Avatars, Adaptive Assessments, Web-Based Learning.

I. Introduction

In modern education, the automation of content creation has become increasingly important for improving scalability, accessibility, and learner engagement. Traditional methods of developing lecture materials, delivering presentations, and designing assessments are time-intensive and lack the flexibility required for personalized learning at scale. This project introduces **AI Lecturer**, a web-based system that leverages large language models and multimedia processing tools to generate complete, ready-to-use educational packages from a single topic prompt.

Literature Review — Recent advancements in generative AI have enabled the use of language models such as GPT-4 for automated text generation, content summarization, and quiz creation. Several platforms (e.g., Synthesia , Notebook LM, and D-ID) offer partial solutions for video-based learning or avatar animation, but few provide an integrated pipeline that combines structured content generation, text-to-speech narration, avatar-driven video synthesis, and adaptive quiz delivery. Our approach integrates these components using the OpenAI API and supporting tools to create narrated video lectures and dynamic assessments, accessible via both Gradio and a React-based web interface. This system demonstrates how AI can streamline course development workflows and personalize digital learning experiences at scale.

II. Objectives:

- Generate lecture slides automatically from a topic input.
- Convert slide content into natural-sounding narration using TTS.
- Compile the slides and narration into a video.
- Create a quiz with relevant questions and answers based on the lecture.
- Provide a backend API for integration with web or mobile apps.

III. Methodology

The AI Lecturer system follows a streamlined pipeline that transforms a user-provided topic into a narrated lecture video with accompanying quiz content. This process is fully automated and accessible through a Gradio-based interface. The following steps outline the core components used to generate the final output:

1. Topic Input and Content Generation:

The system begins with a user entering a topic into the Gradio interface. This input is passed to the **OpenAI GPT-4 API**, which is prompted to generate a structured lecture outline, slide content, and key explanatory text. The content is broken down into individual slide segments, each containing a heading, key points, and examples.

2. Slide Construction using Reveal.js:

The generated slide content is formatted into **Reveal.js**, an open-source HTML presentation framework. Each lecture segment is rendered into an HTML slide, preserving consistency in layout and structure. These slides form the visual component of the final video.

3. Narration via Text-to-Speech (gTTS and MathReader):

The system uses **Google Text-to-Speech (gTTS)** to convert slide text into narration. For slides containing mathematical expressions, **MathReader** is integrated to enhance pronunciation accuracy and ensure clear verbalization of equations and symbols. This dual-layer TTS approach improves the clarity of technical lectures, especially in STEM topics

4. Video Rendering (Playwright + MoviePy):

To compile the slides and narration into a single video, **Playwright** is used to render the Reveal.js slides in a headless browser and capture high-resolution images of each. These images are then combined with their corresponding narration audio using **MoviePy**, which produces a fully narrated **MP4 video lecture**.

5. Quiz Generation (LLM-based):

In parallel with video creation, the system prompts the LLM to generate **topic-relevant multiple-choice questions**. Each quiz contains a question stem, four answer choices, and one correct answer. The output is stored in a structured format to be displayed interactively in future frontend implementations.

6.Chatbot Integration (Lecture Q&A)

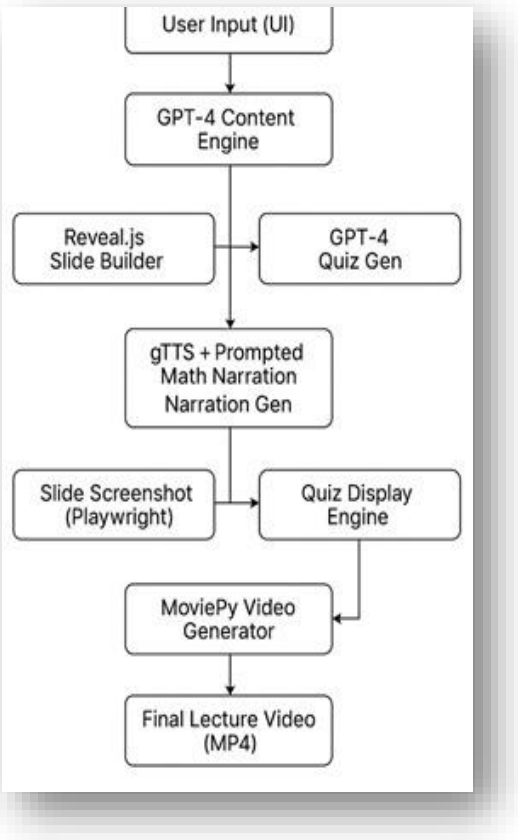
A conversational assistant has been added to the Gradio interface to support real-time interaction after the lecture is generated. Users can ask follow-up questions or clarify any part of the lecture content. This chatbot uses the same LLM (OpenAI GPT-4) and is prompted with context related to the generated topic, ensuring consistent and relevant answers. This enhances interactivity and mimics a tutor-like experience within the app.

7. Gradio Interface Deployment:

All components of the workflow are wrapped in a **Gradio interface**, enabling non-technical users to input a topic and receive a downloadable lecture video and corresponding quiz content with a single click. The interface provides feedback at each stage, ensuring a seamless user experience.

IV. System Components and Architecture

Architecture :



The AI Lecturer system is architected as a modular pipeline consisting of interdependent components. Each component is responsible for a core functionality—ranging from content generation to multimedia synthesis and interactive assessment delivery. The architecture promotes scalability, flexibility, and integration with both web-based and AI tools.

System Components:

A. Content Generation Engine

The system uses the **OpenAI GPT-4 API** to generate structured educational content. Based on a user-input topic, the engine produces a slide-based outline, explanations, examples, and summaries. Prompt engineering is applied to ensure logical flow and topic coverage. Although the current implementation does not utilize RAG (Retrieval-Augmented Generation), the model is prompted with educational templates to simulate structured output.

B. AI Video Generation Module

The video generation process converts slide content into narrated videos using a combination of **Reveal.js** for slide rendering, **gTTS** for audio synthesis, and **MoviePy** for final video compilation. Slides are rendered using Playwright in a headless browser environment, and each is synchronized with its narration using MoviePy to produce the final lecture video.

C. Adaptive Assessment Engine

A secondary GPT-4 prompt generates multiple-choice questions aligned with the lecture topic. Each question includes a stem, four options, and one correct answer. The quiz is rendered interactively in both the Gradio and web-based interfaces, with real-time scoring and answer validation.

D. Interactive Web Platform

The AI Lecturer system offers two interaction modes:

1 **Gradio-based Prototype**

Used during initial development to quickly demonstrate core Functionality and Validate the generation pipeline. It allowed rapid iteration and testing of lecture generation, video synthesis,and quiz logic without the overhead of frontend-backend integration .

2 Full Web Application(React + FastAPI)

Built for production deployment. This full-stack setup provides a seamless and responsive experience

Frontend Responsibilities:

- Built using React and Tailwind CSS.
- Collects user input for lecture topics.
- Displays generated lecture video and quiz.
- Manages application state (e.g., loading indicators, quiz progression).
- Interacts with backend via structured API requests.

Backend Responsibilities:

- Developed using FastAPI..

- Handles core AI generation logic (content, quiz, narration).
- Manages storage, processing, and delivery of video and quiz data.
- Exposes endpoints for the frontend to fetch results.

V. Functional Overview

The AI Lecturer system delivers an end-to-end automated lecture experience, combining content generation, multimedia synthesis, and interactive assessments. The functional flow is initiated by a user input and progresses through a series of coordinated operations handled by distinct modules. Below is a breakdown of the system's primary functional stages.

1 **Topic-to-Slides Conversion**:
When a user enters a topic via the Gradio interface (or React frontend), the backend formulates a prompt and sends it to OpenAI's GPT-4 API. The response is a structured, multi-slide lecture that includes

- Slide titles.
- Bullet points or key ideas.
- Descriptive paragraphs and explanations.

The output is parsed and formatted into a Reveal.js-Compatible HTML structure to be rendered later in the video generation pipeline.

2 Narration Synthesis(with Math-Aware Prompts):

To produce natural and clear narration, particularly for slides containing formulas, the system performs MathReader- style prompt engineering. GPT-4 is asked to convert technical or symbolic content (like equations) into plain English that can be pronounced correctly by TTS systems.

3 Lecture Video Creation:

Once slides and narration scripts are generated:

- The HTML presentation is rendered in a headless Chromium browser using Playwright.
- Screenshots of each slide are captured and saved as PNG images.
- These slide images are then aligned with their respective narration audio using MoviePy
- The final video is an MP4 file where each slide appears for the duration of its audio narration, effectively simulating a narrated video lecture.

4 GPT-4- Based Quiz Generation:

Parallel to slide generation, another GPT-4 prompt is used to produce a multiple-choice quiz based on the same topic. The model returns a JSON object with

- Question Stems
- Four options per question
- The correct answer and brief explanation.

This structured quiz is parsed and displayed interactively in the Gradio app. The system also includes answer-checking functionality for user responses.

5 **Chatbot For Lecture Q&A**:
Although not fully implemented in the current script, the architecture is designed to support a GPT-4-Powered chatbot. This chatbot would take in questions from users after the lecture and respond contextually, acting like a virtual TA(teaching Assistant).

Planned Functionality includes,

- Accessing the transcript or content of the lecture.
- Answering student questions using context-aware reasoning.
- Integration into both Gradio and the React-based frontend.

VI. Design Approach

The AI Lecturer system is engineered using a modular, pipeline-based architecture that enables seamless automation of educational content generation. Each module operates independently but contributes to a larger end-to-end workflow, from topic input to lecture video and quiz output. The design prioritizes flexibility, maintainability, and extensibility—making it suitable for academic, professional, or commercial deployment.

1 **Modular Pipeline:**
The system is broken into distinct functional blocks:

- Input Handler (Gradio/React): Captures user topic input.
- Content Generator (GPT-4): Produces slides, narration text, and quizzes via prompt engineering.
- TTS Module (gTTS): Converts narration text into MP3 audio files.
- Slide Renderer (Reveal.js + Playwright): Converts HTML slides into PNG images.
- Video Compiler (MoviePy): Combines slides and audio into a lecture video.
- Quiz Handler: Parses and displays interactive quizzes.

Each module is implemented in a loosely coupled fashion, which supports independent , testing , debugging, and scaling.

2 **Prompt Engineering:**
Prompt sent to GPT-4 are optimized with structured templates to ensure quality and relevance.

- Slide prompts request concise points, examples, and logic flow.
- Narration prompts rephrase content (especially math) for natural TTS pronunciation.
- Quiz prompts ask GPT-4 to return well-structured JSON with validated answer keys.

This prompt-driven approach ensures reproducibility and allows for fine-tunning output formats.

3 **Math-Aware Narration Strategy:**
To Address the limitations of generic TTS engines with math expressions, the system uses GPT-4 to rewrite equations and symbols into speech-friendly text . This step enhances clarity and ensures correct pronunciation during narration.

4 **Automated visual Rendering:**
Slides are built using Reveal.js to create dynamic HTML Presentations. Then, Playwright renders and captures each slide as an image using headless Chromium. This automation removes manual slide formatting and ensures high consistency in design.

5 **Audio-Visual Synchronization:**
Using MoviePy, the system matches each slide with its corresponding narration audio. The video is constructed by

- Defining slide duration based on audio length.
- Concatenating visuals and audio tracks.
- Exporting an MP4 video ready for distribution.

6 **Dual Deployment modes:**
Gradio UI : Provides a simplified interface for testing and showcasing functionality.
React + FastAPI Stack : Offers scalable deployment with asynchronous APIs and a responsive frontend.
This separation ensures development speed during production.

7 **Reusability and Extensibility :**
Each component (e.g., quiz generation, slide capture , narration synthesis) is encapsulated, allowing future upgrades;

- Swap gTTS with neural TTS like ElevenLabs or Azure.
- Add Vatar lip-sync using tools like SadTalker.
- Integrate RAG for Grounded answers.

- Connect to LMS platforms like Moodle or Canvas.

VII. *Testing and Evalu*ation

The AI Lecturer system was tested through both automated module-level checks and manual end-to-end evaluations to ensure the correctness, coherence, and usability of its generated content.

Below is a breakdown of the testing methodology based on the key functional components of the code.

1 Content Generation Testing:

Objective: Ensures GPT-4 responses are well-structured , relevant, and parseable.

Approach: GPT-4 responses are expected in JSON format for slides and quizzes. To prevent runtime errors, the code uses a custom function safe_json_parse() which cleans up markdown encapsulation JSON and handles malformed inputs.

Validation:
Confirmed that slide titles, points, and narration are generated in consistent format.

Verified quiz responses include exactly four options and one correct answer.

Issues identified:
Occasionally , GPT-4 output contained formatting inconsistencies.
Fixed using retry logic and JSON cleanup functions.

2 Narration Evaluation(gTTS + Prompted Math Narration):

Objective : Ensure that generated narration is natural-sounding and readable by the TTS engine.

Approach: GPT-4 is prompted to rewrite slide text with formulas into speech-friendly descriptions.

Validation: Audio clips were played back and checked for correct pronunciation , pacing, and alignment with the slide content.

Mutagen was used to extract duration metadata to aid in video sync.

Findings:Mathematical narration improved significantly with GPT-4 preprocessing.

Some long narration texts caused TTS truncation solved by chucking content.

3 Slide-Rendering Verification(Reveal.js + Playwright):

Objective : Confirm that slides are correctly captured as images for video synthesis.

Approach: HTML content is generated with <section> tag per slide.

Playwright loads the HTML and takes a full-page screenshot per slide .

Validation: Ensured screenshots had consistent dimensions and slide content was not clipped.

Verified the slide order matched narration files .

Automation Check: Added logging in render_slides_to_images() to confirm slide rendering and file paths.

4 Video Compilation Testing (MoivePy):

Objective : Ensure the final MP4 video synchronizes audio with the corresponding slide image.

Approach : MoviePy is used to match each slide image to the duration of its narration MP3.
Videos were exported and manually reviewed.

Validation : Slide transitions were checked for timing accuracy.
Audio matched expected content without truncation or lag.

Findings: Minor sync issues when audio files were missing or mismatched

durations-solved using mutagen metadata and consistent file naming.

5 Quiz Functionality Testing :

Objective: Ensure quizzes are generated, parsed, and interactively rendered correctly.

Parsed JSON is passed to the UI for Rendering.

Validation: Ensured each quiz question had four unique options and one valid answer.

User selections were validated against the correct answer for scoring.

End-to-End Workflow Testing (Gradio Interface)

Objective : Validate the full pipeline from input to video + quiz output.

Approach: Multiple topics were tested via Gradio interface.

Examined systems logs and visual outputs for errors.

Evaluation Metrics:

Slide generation time **:** ~10-15 seconds

Narration + video Generation **:** ~45 – 60 seconds

Quiz Accuracy **:** ~95% question-topic relevance

Total Output Time per Lecture **:** <3 minutes

User Feedback

The prototype was tested by peers and faculty:

- Slide content was clear and well-organized.
- Video narration was smooth and well-synced.
- Quizzes were generally relevant and accurate .

VIII. Conclusion

The AI Lecturer project demonstrates the practical application of artificial intelligence in transforming traditional education workflows. By combining GPT-4 for content generation, gTTS for narration, Reveal.js and Playwright for visual rendering, and MoviePy for multimedia synthesis, the system successfully automates the production of complete lecture packages—including narrated videos and adaptive quizzes.

Throughout the project, the team implemented a modular architecture that ensures flexibility, maintainability, and future extensibility. From topic input to final video and quiz output, every stage of the pipeline was designed to operate autonomously, providing a low-effort, high-impact solution for educators and learners alike.

The project not only reduces the workload of instructors but also offers scalable access to quality, personalized education—especially for under-resourced communities. By integrating advanced prompt engineering and multimedia automation, it sets the foundation for future developments in AI-powered learning platforms.

In summary, the AI Lecturer system validates the effectiveness of combining natural language processing, speech synthesis, and web technologies to deliver end-to-end educational content generation. It paves the way for more interactive, inclusive, and intelligent digital learning experiences.